SOFTWARE ENGINEERING LAB

LAB 7: FUZZ TESTING FOR INPUT VALIDATION BUGS

Student Name	Purvik V
SRN	PES1UG24AM211
Experiment	Fuzz testing of a Smart Thermostat Controller

Objective

To explore fuzz testing and input validation by testing a Smart Thermostat Controller's input-processing functions, identifying failures caused by invalid inputs, applying validation in the corrected implementation, and verifying the behaviour using Hypothesis and pytest.

Functions tested

1. **parse_setpoint()** – parses a temperature setpoint expressed in Celsius or Fahrenheit.
2. **average_temperature()** – calculates the average of comma-separated sensor readings.
3. **schedule_slot()** – interprets a time in HH:MM format and labels it as a night or day slot.

Tools and requirements

Python 3.15.2, pytest 9.2.0, and Hypothesis 6.170.15 on Windows PowerShell. The test run was performed with `python -m pytest -v test_thermostat_control.py`.

1. Original Application – Bugs Identified

The original starter program did not validate all user inputs before attempting to convert or process them. The screenshots below document the observed failures and invalid input acceptance.

1.1 Invalid temperature setpoint

Input: **abc**. The original `parse_setpoint()` attempted to convert the text to a floating-point number and raised a `ValueError`.

```
PS C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7> cd starter
PS C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7\starter> python therm
ostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): abc
Traceback (most recent call last):
  File "C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7\starter\thermost
at_control.py", line 46, in <module>
    main()
    ~~~~^^
  File "C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7\starter\thermost
at_control.py", line 31, in main
    result = parse_setpoint(user_input)
  File "C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7\starter\thermost
at_control.py", line 9, in parse_setpoint
    value = float(text[:-1])
ValueError: could not convert string to float: 'ab'
```

Figure 1. Original application fails when a non-numeric setpoint is entered.

1.2 Invalid time accepted

Input: **25:70**. The original `schedule_slot()` accepted an out-of-range hour and minute and returned **day 70** instead of rejecting the invalid time.

```
PS C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\
PES1UG24AM211 SE LAB 7\starter> python thermostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): 21.5C
Setpoint in Celsius: 21.5

==== Running average_temperature ====
Enter sensor readings as CSV (e.g. 20.5,21,22.5): 20,21,22
Average Temperature: 21.0

==== Running schedule_slot ====
Enter a time as HH:MM (e.g. 07:30): 25:70
Schedule Slot: day 70
PS C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\
PES1UG24AM211 SE LAB 7\starter>
```

Figure 2. Original application accepts the invalid time 25:70.

1.3 Empty sensor reading

Input: **20.5,,22.5**. Splitting the CSV string produced an empty element. Converting that element to a number raised a `ValueError`.

```
PS C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7\starter> python therm
ostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): 21.5C
Setpoint in Celsius: 21.5

==== Running average_temperature ====
Enter sensor readings as CSV (e.g. 20.5,21,22.5): 20.5,,22.5
Traceback (most recent call last):
  File "C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7\starter\thermost
at_control.py", line 46, in <module>
    main()
    ~~~~^^
  File "C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7\starter\thermost
at_control.py", line 36, in main
    result = average_temperature(user_input)
  File "C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7\starter\thermost
at_control.py", line 16, in average_temperature
    values = [float(r) for r in readings_csv.split(",")]
              ~~~~~^^^
ValueError: could not convert string to float: ''
```

Figure 3. Original application fails on an empty CSV sensor reading.

2. Corrected Application – Successful Execution

The corrected application was executed with valid inputs: setpoint **21.5C**, sensor readings **20.5,21,22.5**, and time **07:30**. The program displayed 21.5 Celsius, an average temperature of approximately 21.3333, and the schedule slot **day 30**.

```
PS C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\
PES1UG24AM211 SE LAB 7> python thermostat_control.py
==== Running parse_setpoint ====
Enter a setpoint (e.g. 21.5C or 70F): 21.5C
Setpoint in Celsius: 21.5

==== Running average_temperature ====
Enter sensor readings as CSV (e.g. 20.5,21,22.5): 20.5,21,22.5
Average Temperature: 21.333333333333332

==== Running schedule_slot ====
Enter a time as HH:MM (e.g. 07:30): 07:30
Schedule Slot: day 30
PS C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\
PES1UG24AM211 SE LAB 7>
```

Figure 4. Successful execution of the corrected thermostat application.

3. Hypothesis Fuzz Testing and Results

The test suite used Hypothesis-generated inputs and explicit invalid-input cases to exercise the three functions. It checked that unexpected exceptions were not leaked, invalid values were rejected, valid conversions and calculations worked, and schedule boundary cases behaved as expected.

Test coverage shown in the run

• Setpoint parsing: arbitrary text, None, invalid units and formats, Celsius round-trip, and Fahrenheit conversion.
• Average calculation: arbitrary text, None, empty readings, malformed CSV entries, and comparison with Python's mean calculation.
• Schedule parsing: arbitrary text, None, malformed time strings, out-of-range hour/minute, and night/day boundary checks.

```
============================= test session starts =============================
platform win32 -- Python 3.15.2, pytest-9.2.0, pluggy-1.7.0 -- C:\Python315\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\Purvik v gowda\OneDrive\Desktop\SEM-5 LABS\SE LAB\PES1UG24AM211\PES1UG24AM211 SE LAB 7
plugins: hypothesis-6.170.15
collected 29 items

test_thermostat_control.py::test_parse_setpoint_never_leaks_unexpected_exceptions PASSED [  3%]
test_thermostat_control.py::test_parse_setpoint_rejects_none PASSED  [  6%]
test_thermostat_control.py::test_average_temperature_never_leaks_unexpected_exceptions PASSED [ 10%]
test_thermostat_control.py::test_average_temperature_rejects_none PASSED [ 13%]
test_thermostat_control.py::test_schedule_slot_never_leaks_unexpected_exceptions PASSED [ 17%]
test_thermostat_control.py::test_schedule_slot_rejects_none PASSED   [ 20%]
test_thermostat_control.py::test_celsius_round_trip PASSED           [ 24%]
test_thermostat_control.py::test_average_matches_python_mean PASSED  [ 27%]
test_thermostat_control.py::test_schedule_valid_time PASSED          [ 31%]
test_thermostat_control.py::test_invalid_setpoints[] PASSED          [ 34%]
test_thermostat_control.py::test_invalid_setpoints[21] PASSED        [ 37%]
test_thermostat_control.py::test_invalid_setpoints[21K] PASSED       [ 41%]
test_thermostat_control.py::test_invalid_setpoints[abcC] PASSED      [ 44%]
test_thermostat_control.py::test_invalid_setpoints[21Cjunk] PASSED   [ 48%]
test_thermostat_control.py::test_invalid_readings[] PASSED           [ 51%]
test_thermostat_control.py::test_invalid_readings[1,,2] PASSED       [ 55%]
test_thermostat_control.py::test_invalid_readings[a,2] PASSED        [ 58%]
test_thermostat_control.py::test_invalid_readings[,] PASSED          [ 62%]
test_thermostat_control.py::test_invalid_readings[1,] PASSED         [ 65%]
test_thermostat_control.py::test_invalid_times[] PASSED              [ 68%]
test_thermostat_control.py::test_invalid_times[7:3] PASSED           [ 72%]
test_thermostat_control.py::test_invalid_times[24:00] PASSED         [ 75%]
test_thermostat_control.py::test_invalid_times[12:60] PASSED         [ 79%]
test_thermostat_control.py::test_invalid_times[1:02] PASSED          [ 82%]
test_thermostat_control.py::test_invalid_times[ab:cd] PASSED         [ 86%]
test_thermostat_control.py::test_invalid_times[12-30] PASSED         [ 89%]
test_thermostat_control.py::test_fahrenheit_conversion PASSED        [ 93%]
test_thermostat_control.py::test_schedule_night_boundary PASSED      [ 96%]
test_thermostat_control.py::test_schedule_day_boundary PASSED        [100%]

============================= 29 passed in 6.67s ==============================
```

Figure 5. Pytest output showing all 29 tests passed.

Result	Value
Tests collected	29
Tests passed	29
Tests failed	0
Execution time	6.67 seconds

4. Conclusion

The original program demonstrated input-validation weaknesses in temperature parsing, CSV sensor-reading processing, and time validation. The corrected implementation executed successfully with valid sample inputs. The pytest run collected and passed all 29 tests in 6.67 seconds, providing evidence that the implemented validation and expected behaviours passed the test suite.

Submitted files: corrected `thermostat_control.py`, `test_thermostat_control.py`, `requirements.txt`, and the original starter files.